INTELLIGENT AIRPORT RUNWAY SCHEDULING AND CONFLICT MANAGEMENT SYSTEM USING AVL TREES

CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0309 – DATA STRUCTURES

to the award of the degree of

BACHELOR OF TECHNOLOGY IN

(AI&DS)

Submitted by

Pujitha K (192524149)

Under the Supervision of

Dr. UMA PRIYADARSINI P.S

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled " Intelligent Airport Runway Scheduling and Conflict Management system using AVL Trees " has been carried out by Pujitha K (192524149) and under the supervision of Dr.Uma Priyadarsini P.S and is submitted in partial fulfilment of the requirements for the current semester of the B.Tech (AI&DS) program at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

Dr.Kiruthika K
Professor
CSE (Engineering)
SIMATS Engineering,
SIMATS, Chennai – 602105.

COURSE FACULTY

Dr. Uma Priyadarsini P.S
Professor
CSE (Deep Learning)
SIMATS Engineering,
SIMATS, Chennai – 602105.

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

DECLARATION

We, Pujitha K (192524149) of the B.tech (AI&DS), SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "Intelligent Airport Runway Scheduling and Conflict Management system using AVL Trees" is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:09-09-2026

Signature of the Students with Names
Pujitha K (192524149)

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
Pujitha K 192524149	AVL and scheduling	AVL module development	System testing	Report preparation	40%
Sathwika Reddy 192525274	System design	Scheduling module development	Test case preparation	Literature review	30%
Rohini Sree 192524471	Testing and documentation	System integration	Result analysis	Report formatting	30%
Total					**100%**

ABSTRACT

Airports handle a large number of aircraft arriving, departing, and moving through runways within limited time intervals. Efficient runway scheduling is an important engineering problem because improper allocation of runways or overlapping aircraft schedules can lead to runway conflicts, delays, increased fuel consumption, and safety risks. Traditional scheduling methods may become inefficient when the number of aircraft increases, making fast searching, insertion, deletion, and conflict detection essential for real-time airport operations.

The proposed Intelligent Airport Runway Scheduling and Conflict Management System using AVL Trees provides an efficient data-structure-based approach for managing aircraft runway schedules. The system is designed to maintain aircraft scheduling information such as aircraft ID, arrival time, departure time, runway number, and priority. AVL trees are used to store and organize the scheduling records while maintaining a balanced structure. This enables efficient insertion, deletion, searching, and updating of aircraft records. Before assigning a runway, the system checks the scheduled time intervals to identify possible conflicts and prevents overlapping runway usage. Priority-based scheduling can also be incorporated to handle emergency or high-priority aircraft efficiently.

The methodology involves collecting aircraft scheduling details, storing them in an AVL tree, balancing the tree after insertion or deletion, searching for available scheduling slots, detecting time-based runway conflicts, and assigning suitable runways. The system can also update or remove completed schedules while maintaining the balance of the AVL tree. Performance is evaluated based on scheduling efficiency, search time, conflict detection, and the ability to handle increasing numbers of aircraft records.

The major results demonstrate that AVL-tree-based scheduling provides fast data access and maintains efficient operations even as the number of aircraft increases. The balanced tree structure reduces the time required for searching, insertion, and deletion compared with an unbalanced binary search tree. Conflict detection further improves runway utilization and reduces the possibility of scheduling overlaps.

In conclusion, the proposed system demonstrates how AVL trees can be effectively applied to intelligent airport runway scheduling. By combining balanced-tree data management with conflict

detection and priority-based scheduling, the system provides a reliable, scalable, and efficient approach for improving airport runway management and operational safety.

Keywords: AVL Tree, Airport Runway Scheduling, Conflict Detection, Aircraft Management, Data Structures, Priority Scheduling

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AVL	Adelson-Velsky and Landis	3
BST	Binary Search Tree	3
FCFS	First Come, First Served	2

CHAPTER 1
INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

Airports are complex transportation systems where aircraft must be managed safely and efficiently during arrival, departure, taxiing, and runway operations. Runways are limited resources, and multiple aircraft may require access to the same runway within overlapping time intervals. Therefore, efficient runway scheduling is essential to maintain safe aircraft separation, reduce delays, and improve airport capacity.

Traditional scheduling approaches may rely on sequential searching or manually maintained records, which can become inefficient when the number of aircraft increases. An airport scheduling system requires frequent operations such as inserting new aircraft schedules, searching for existing records, updating schedules, deleting completed flights, and identifying conflicts between aircraft. These operations require an efficient data structure capable of maintaining fast access while handling dynamic scheduling information.

An AVL tree, which is a self-balancing binary search tree, can be used to efficiently manage aircraft scheduling records. It automatically maintains balance after insertion and deletion, providing approximately O(log n) time complexity for searching, insertion, and deletion. By combining AVL tree operations with time-interval checking and runway allocation, an intelligent scheduling system can detect potential conflicts and support efficient runway management.

1.2 Need for the Project

The increasing number of aircraft operations creates a need for efficient runway resource management. If two or more aircraft are assigned to the same runway during overlapping time intervals, runway conflicts may occur, potentially resulting in delays and safety risks.

The problem affects airport authorities, air traffic controllers, airline operators, pilots, and passengers. Delays in runway scheduling can cause flight delays, inefficient runway utilization, increased fuel consumption, and operational difficulties.

Existing basic scheduling methods may become inefficient when handling a large number of dynamically changing aircraft records. Sequential searching can require considerable time as the dataset grows, while an unbalanced binary search tree may also become inefficient in the worst case.

From an engineering perspective, the project is significant because it applies an efficient data structure to a real-world resource-allocation problem. The combination of AVL trees, scheduling logic, conflict detection, and priority handling provides a systematic approach to improving the efficiency and reliability of runway management.

1.3 Problem Statement

The engineering problem is to design and implement an intelligent airport runway scheduling system capable of managing multiple aircraft with different arrival times, departure times, runway requirements, and priorities while preventing scheduling conflicts.

The system must efficiently handle dynamic aircraft records and perform insertion, searching, updating, and deletion operations while maintaining balanced data storage. It must identify overlapping time intervals on the same runway and allocate suitable runway slots without violating scheduling constraints.

The system must therefore balance multiple requirements, **including** fast data access, conflict-free scheduling, efficient runway utilization, priority handling, and scalability under realistic constraints such as limited runway availability and continuously changing aircraft schedules.

1.4 Project Aim

The aim of this project is to design and implement an intelligent airport runway scheduling and conflict management system using AVL trees to efficiently manage aircraft schedules, detect runway conflicts, and improve runway resource utilization.

1.5 Project Objectives

The main objectives of the project are:

1. To design an efficient data model for storing aircraft scheduling information.
2. To develop an AVL tree-based structure for managing aircraft records efficiently.
3. To implement insertion, deletion, searching, and updating operations for aircraft schedules.
4. To detect conflicts by comparing aircraft time intervals and runway assignments.
5. To implement suitable runway allocation based on availability and scheduling constraints.
6. To incorporate priority handling for aircraft requiring urgent or preferred scheduling.
7. To test the system using different numbers of aircraft and scheduling conditions.

8. To evaluate the efficiency of AVL-tree operations and the effectiveness of conflict detection.

1.6 Scope

Included

The project includes:

- Storage and management of aircraft scheduling records.
- Aircraft ID, arrival time, departure time, runway number, and priority information.
- AVL tree implementation for efficient record management.
- Insertion, deletion, searching, and updating of aircraft records.
- Runway availability checking.
- Time-interval-based conflict detection.
- Basic priority-based scheduling.
- Display of scheduled aircraft and detected conflicts.
- Performance evaluation based on search, insertion, deletion, and conflict-management operations.

Excluded

The project does not include:

- Real-time communication with actual aircraft.
- Direct integration with airport radar or air traffic control systems.
- Automatic control of aircraft movement.
- Weather prediction and detailed weather-based scheduling.
- Real-world pilot communication.
- Full-scale airport management and commercial flight databases.

Assumptions

The system assumes that:

- Aircraft scheduling information is available as input.
- Arrival and departure times are provided correctly.
- Runway information is known to the system.
- Each aircraft requires a defined runway time interval.

- The number of available runways is predetermined.
- Priority values are assigned according to predefined rules.

Boundary Conditions

The system operates within the following boundaries:

- Only the runways defined in the system can be assigned.
- Two aircraft cannot occupy the same runway during overlapping time intervals.
- Aircraft schedules must contain valid time information.
- The AVL tree must remain balanced after insertion and deletion.
- Scheduling decisions are based on the available input data and predefined rules.

1.7 Expected Engineering Outcome

The expected engineering outcome is a software prototype and algorithmic solution for intelligent airport runway scheduling and conflict management.

The final system is expected to provide an AVL-tree-based scheduling module that efficiently stores and retrieves aircraft information, performs dynamic schedule updates, identifies runway conflicts, and assists in selecting suitable runway slots.

The prototype should demonstrate improved data-management efficiency through balanced-tree operations and provide reliable conflict detection for overlapping runway schedules. The project is intended to serve as **a** proof-of-concept engineering solution demonstrating how data structures and scheduling algorithms can be applied to a real-world airport resource-management problem.

CHAPTER 2
LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

The literature review was conducted by identifying research papers, aviation guidelines, existing airport scheduling technologies, and data-structure-based scheduling methods related to airport runway scheduling, aircraft sequencing, conflict detection, runway utilization, and real-time resource management. Relevant academic studies were reviewed from research databases and technical publications, while operational requirements were examined using guidance and information published by aviation authorities and organizations such as the Federal Aviation Administration (FAA) **and** EUROCONTROL**.**

The review focused mainly on recent research involving mathematical optimization, heuristic methods, collaborative decision-making, real-time scheduling, and intelligent algorithms. Particular attention was given to identifying the data-management requirements of runway scheduling and determining whether balanced search-tree structures such as AVL trees have been directly applied to this problem.

2.2 Review of Existing Work

2.2.1 Aircraft Runway Scheduling

Aircraft runway scheduling involves determining the sequence and operating times of aircraft while satisfying operational constraints. A major survey of the aircraft runway scheduling problem identifies exact methods such as mixed-integer programming and dynamic programming, along with stochastic approaches, metaheuristics, and reinforcement learning. The problem is considered computationally difficult because several operational constraints must be satisfied simultaneously.

These approaches can produce high-quality schedules, but optimization models may require significant computational resources when the number of aircraft and constraints increases. This creates a need for efficient data-management mechanisms for rapidly accessing and updating scheduling information.

2.2.2 Collaborative Airport Scheduling

Research on collaborative decision-making has proposed mathematical models for rescheduling arrivals and departures. Such approaches consider different objectives, including efficiency, fairness, and on-time performance, while recognizing that airlines, airports, and air-navigation service providers may have different requirements.

This demonstrates that airport scheduling is not simply a matter of assigning the next available runway. Multiple stakeholders and competing objectives must be considered.

2.2.3 Airspace and Taxiway Coordination

Samà et al. studied coordination between airborne operations and taxiway scheduling. Their work highlights that weak coordination can produce runway queues, increased aircraft delays, and higher energy consumption. Mathematical and heuristic approaches were evaluated using practical-size airport instances.

This work is relevant because it shows the importance of considering interactions between different airport operations rather than treating runway scheduling as an isolated task.

2.2.4 Operational Runway Separation and Conflict Management

Operational airport procedures require aircraft to be appropriately separated to avoid conflicts. FAA guidance includes specific procedures for intersecting and converging runway operations, demonstrating that runway conflict management depends on factors such as aircraft position, runway configuration, and separation conditions.

EUROCONTROL has also investigated runway occupancy time and optimized spacing to improve runway capacity while maintaining safety requirements.

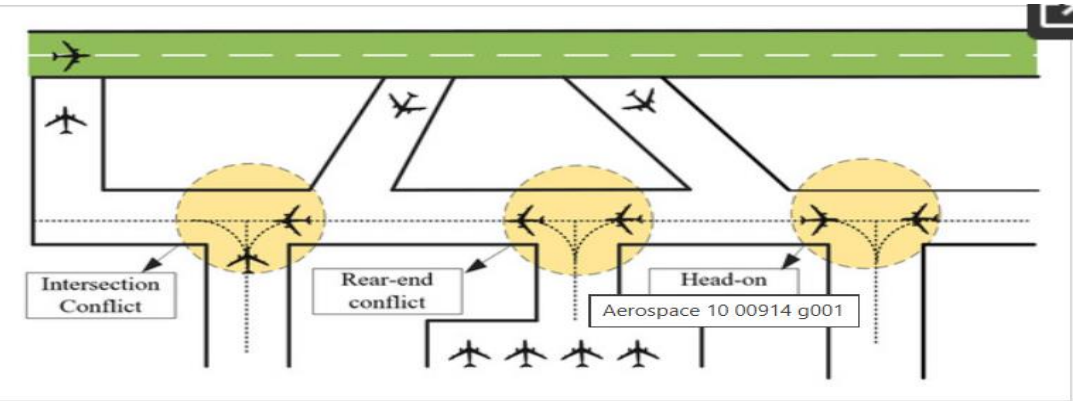


Figure 1. Taxiing conflict types.

2.2.5 Recent Intelligent Scheduling Approaches

Recent research has moved toward intelligent and integrated scheduling. A 2026 study proposed a model-based Monte Carlo Tree Search approach that coordinates runway, taxiway, apron, and airspace resources. Experiments using real airport data showed improvements over selected metaheuristic and policy-based approaches.

Another recent approach uses multi-objective optimization to balance punctuality, runway workload, taxiing time, and environmental objectives.

These methods demonstrate the potential of intelligent scheduling but also indicate the increasing complexity of airport scheduling systems.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Manual / Controller-Based Scheduling	Uses operational experience; flexible for unusual situations	Time-consuming; difficult to manage large numbers of schedules; human errors possible	Provides the basic scheduling problem that the proposed system aims to computerize
First-Scheduled, First-Served (FSFS)	Simple and relatively fair; easy to implement	Does not always provide optimal runway utilization; limited conflict optimization	Can be used as a baseline scheduling method
Mathematical Optimization / MILP	Can model multiple constraints and objectives; produces optimized solutions	Computationally expensive for large and complex problems	Provides advanced scheduling concepts against which the prototype can be compared
Heuristic / Metaheuristic Methods	Suitable for complex scheduling; can find good solutions within limited time	Solution quality may depend on parameters; no guarantee of optimality	Demonstrates the need for efficient but simpler scheduling mechanisms
Collaborative	Considers multiple	More complex	Inspires priority and

Decision-Making Systems	stakeholders and operational objectives	implementation; requires integration of multiple data sources	constraint-based scheduling in the proposed system
Runway Occupancy / Separation Methods	Improves safety and runway capacity; considers operational separation	Primarily focused on aircraft separation and runway occupancy rather than data management	Provides rules for conflict detection and safe scheduling
Advanced Intelligent Scheduling	Handles multiple resources and dynamic conditions	Requires sophisticated models, large datasets, and higher computational complexity	Shows the future direction of intelligent airport scheduling
Proposed AVL-Tree-Based System	Fast search, insertion, deletion, balanced data storage, simple conflict checking	Prototype does not model the complete airport environment	Combines efficient AVL-tree data management with runway scheduling and conflict detection

Table 1 : Comparative analysis

2.4 Research / Engineering Gap

The reviewed studies demonstrate that airport runway scheduling has been widely investigated using mathematical programming, heuristic algorithms, collaborative decision-making, and intelligent optimization techniques. However, many advanced approaches focus primarily on finding an optimized aircraft sequence rather than on the underlying data structure used for continuously managing dynamic scheduling records.

Existing advanced solutions can also become complicated when multiple resources such as runways, taxiways, aprons, and airspace are considered simultaneously. For an educational and engineering prototype, there is a need for a simpler approach that can demonstrate efficient schedule storage, rapid searching, dynamic record updates, and basic runway conflict detection.

A specific gap identified for the proposed work is the limited focus on using a self-balancing AVL tree as the core data structure for dynamic airport runway schedule management. AVL trees provide logarithmic-time search, insertion, and deletion in the balanced case, making them suitable for frequently changing scheduling records.

Therefore, the engineering gap is to develop a lightweight scheduling system that combines balanced data management with time-based runway conflict detection, without requiring the computational complexity of a full-scale airport optimization system.

2.5 Proposed Contribution

The proposed project contributes an AVL-tree-based intelligent airport runway scheduling and conflict management prototype. Unlike approaches that concentrate mainly on mathematical optimization of the complete airport system, the proposed work focuses on efficient management of dynamic scheduling records.

The system stores aircraft information such as aircraft ID, arrival time, departure time, runway assignment, and priority in an AVL tree. The balanced structure allows efficient searching, insertion, deletion, and updating of records. Before assigning a runway, the system checks the time interval of existing aircraft schedules to identify possible conflicts.

The main contribution is therefore the integration of:

- **AVL tree** for efficient schedule management.
- **Time-interval checking** for runway conflict detection.
- **Runway availability checking** for resource allocation.
- **Priority-based scheduling** for handling important aircraft.
- **Performance evaluation** using search, insertion, deletion, and conflict-detection operations.

The proposed system is not intended to replace certified air-traffic-control systems. Instead, it provides **a** proof-of-concept engineering model demonstrating how an efficient data structure can support intelligent runway scheduling and conflict management.

Figure 1 : Types of runways in Aviation

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

3.1.1 Stakeholder / User Requirements

Stakeholder	Requirement
Air Traffic Controller / Scheduler	The system should allow aircraft schedules to be added, searched, updated, and removed efficiently.
Airport Operations Staff	The system should provide clear runway allocation information and identify scheduling conflicts.
Airline Operators	The system should support priority-based scheduling and reduce unnecessary scheduling delays.
System Administrator	The system should maintain consistent and valid scheduling records and support reliable data management.
Project Evaluator / Engineer	The system should demonstrate efficient AVL-tree operations and measurable conflict-detection performance.

Table 2 : Stakeholder / User Requirements

3.1.2 Functional & Non-Functional Requirements

Requirement	Type	Measurable Target
Add a new aircraft schedule	Functional	Record inserted successfully into AVL tree
Search for an aircraft	Functional	Search completed in approximately O(log n) for balanced AVL tree
Delete completed/cancelled schedule	Functional	Record removed and tree rebalanced
Update aircraft schedule	Functional	Updated information reflected without duplicate records
Detect runway time conflicts	Functional	100% of intentionally created overlapping test cases detected
Assign available runway	Functional	System should reject conflicting runway slots
Handle aircraft priority	Functional	Higher-priority aircraft considered before lower-priority aircraft when scheduling rules require it
Maintain AVL balance	Non-	Balance factor of every node maintained between -1

	Functional	and +1
System response	Non-Functional	Scheduling operations should complete within an acceptable response time for the test dataset
Reliability	Non-Functional	No loss or duplication of records during normal operations
Usability	Non-Functional	User should be able to enter and view schedules through a simple interface
Scalability	Non-Functional	System should support increasing aircraft records without significant degradation in tree operations

3.2 Constraints & Applicable Standards

The project is a software prototype and not a certified airport operational system. Therefore, aviation standards are used as design references rather than claiming that the prototype is compliant for real-world air-traffic-control deployment.

ICAO Annex 14, Volume I provides Standards and Recommended Practices concerning aerodrome design and operations. FAA runway-safety guidance defines runway incursions and provides safety-oriented procedures and mitigation approaches.

Constraints

- The system is limited to software-based runway scheduling.
- It does not directly control aircraft or runway equipment.
- Input data such as aircraft ID, arrival time, departure time, runway and priority are assumed to be valid.
- The number of runways is predetermined.
- Real-time radar, weather and air-traffic-control data are outside the prototype scope.
- The system cannot replace certified air traffic control or airport safety systems.
- Scheduling decisions are based on predefined rules and available input data.

Applicable Standards / Codes

Standard / Code	Requirement / Relevant Principle	How Applied in This Project
ICAO Annex 14,	Provides Standards and	Used as an aviation safety and

Volume I – Aerodromes	Recommended Practices related to aerodrome design and operations.	operational reference while defining the project's runway-management assumptions.
FAA Order 7050.1 – Runway Safety Program	Provides definitions and procedures associated with runway incursions and surface incidents.	Used as a reference for the project's conflict-management concept and runway-conflict terminology.
FAA AC 150/5300-13 – Airport Design	Provides airport design guidance including runway safety-area considerations.	Used only as background reference; physical runway design is outside the project scope.

Important: These standards are reference inputs for the prototype. The developed system is not claimed to be suitable for direct deployment in a certified airport environment.

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Data structure	Self-balancing AVL tree	—	High
AVL balance factor	-1, 0 or +1	—	High
Search complexity	O(log n) approximately	Time complexity	High
Insertion complexity	O(log n) approximately	Time complexity	High
Deletion complexity	O(log n) approximately	Time complexity	High
Aircraft record	Aircraft ID, arrival, departure, runway, priority	Record	High
Runway conflict	Overlapping intervals on same runway must be rejected	Boolean	Critical
Runway availability	Only available runway/time slots may be assigned	Boolean	Critical
Duplicate aircraft ID	Duplicate IDs should be rejected or handled	Boolean	High
Priority handling	Higher-priority aircraft considered according to	—	Medium

	scheduling rules		
Data consistency	No invalid or contradictory schedule records	—	High
Interface	Simple user input and schedule display	—	Medium
Scalability	Tested with increasing numbers of aircraft	Number of records	High

3.4 Alternative Solutions & Evaluation

Alternative 1: Unbalanced Binary Search Tree

An ordinary Binary Search Tree (BST) can store aircraft schedules according to a selected key such as aircraft ID or scheduled time. It is simple to implement, but its performance can degrade to O(n) when the tree becomes highly unbalanced.

Alternative 2: AVL Tree

An AVL tree is a self-balancing Binary Search Tree that automatically performs rotations after insertion and deletion. It maintains a height difference of at most one between the left and right subtrees, providing approximately O(log n) search, insertion and deletion operations.

Alternative 3: Linear Array/List

A simple array or list can store aircraft scheduling records and is easy to implement. However, searching and inserting records can become inefficient as the number of aircraft increases, particularly when records must be searched sequentially.

Evaluation Matrix

Rating: 1 = Poor, 3 = Moderate, 5 = Excellent.

Criterion	Weight	Alternative 1: BST	Alternative 2: AVL Tree	Alternative 3: Array/List
Performance	30%	3	**5**	2
Cost / Implementation effort	15%	5	4	**5**
Safety / Conflict management support	20%	3	**5**	3

Sustainability / Resource efficiency	10%	3	**4**	2
Reliability	25%	3	**5**	3
Weighted Score	**100%**	**3.35/5**	**4.65/5**	**2.85/5**

Table 5 : Weighted Evaluation of Alternative Data Structures

Selected Alternative

Based on the evaluation, the **AVL tree** is selected because it provides the best balance between performance, reliability, conflict-management support and implementation complexity.

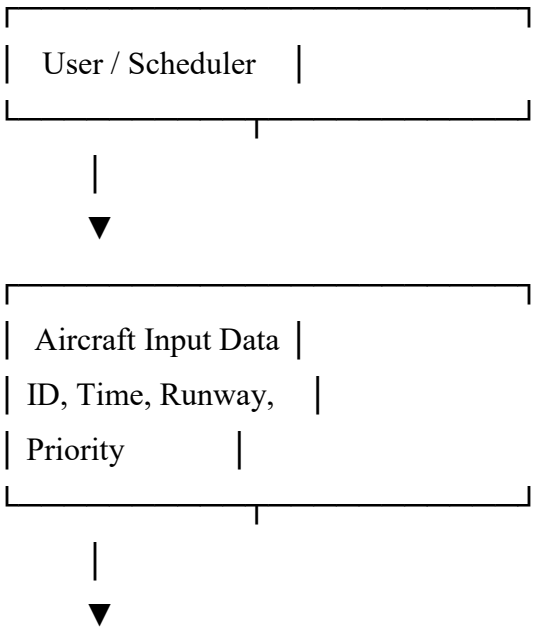
3.5 Selected Design & Detailed Design

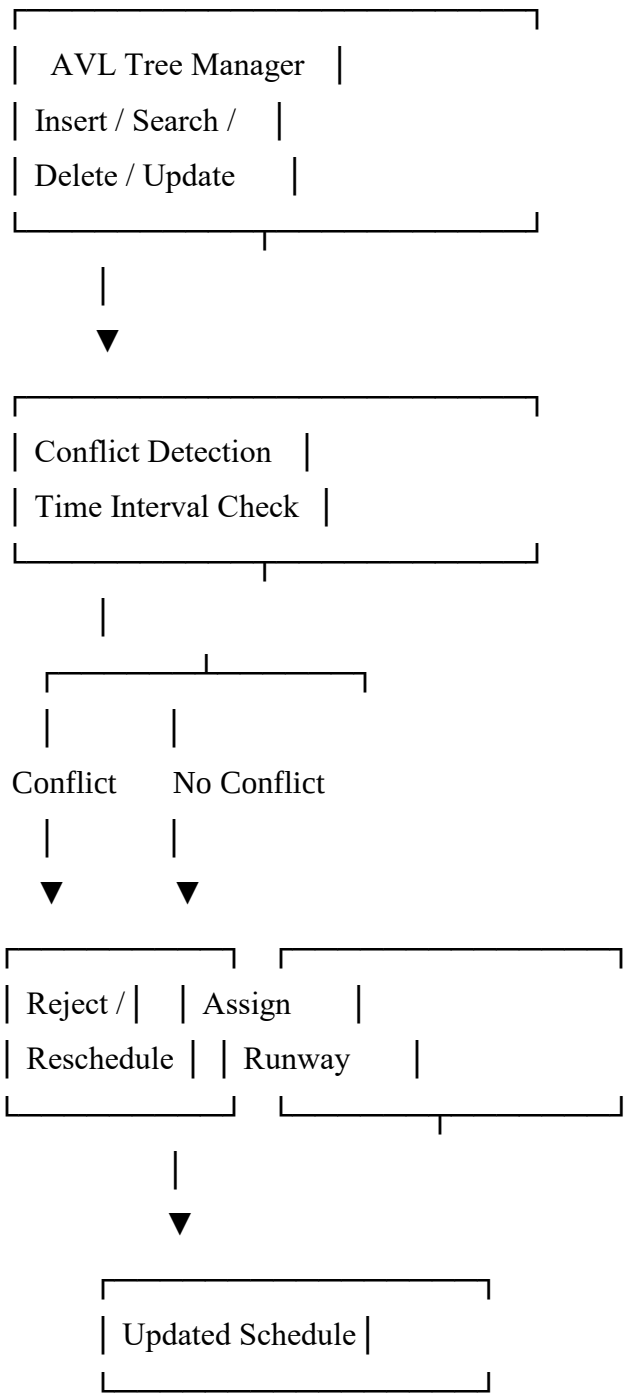
3.5.1 Final Design Selection

The **AVL-tree-based architecture** is selected because runway scheduling requires frequent insertion, searching, updating and deletion of aircraft records. An ordinary BST can become unbalanced, while a simple list may require sequential searching. The AVL tree automatically maintains balance and provides approximately O(log n) operations.

The design combines the AVL tree with a **runway conflict-detection module**. Aircraft schedules are stored using a selected scheduling key, while time-interval checking determines whether a requested runway slot overlaps with an existing schedule.

Proposed System Architecture





Key Engineering Calculation

For an AVL tree, the balance factor is:

Balance Factor = Height(Left Subtree) − Height(Right Subtree)

The acceptable values are:

−1, 0, +1

If the balance factor becomes less than −1 or greater than +1 after insertion or deletion, an appropriate AVL rotation is performed.

The four possible rotation cases are:

- **LL → Right Rotation**
- **RR → Left Rotation**
- **LR → Left Rotation + Right Rotation**
- **RL → Right Rotation + Left Rotation**

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The project was developed using an iterative software development approach. First, the aircraft scheduling requirements and AVL-tree data structure were designed, followed by implementation of AVL insertion, deletion, searching, balancing and rotation operations. The runway scheduling and conflict-detection modules were then integrated with the AVL tree and tested using different aircraft schedules and runway conditions. Finally, the system was evaluated using functional test cases and performance measurements.

Resource	Purpose
Aircraft scheduling dataset	Provides aircraft ID, arrival time, departure time, runway and priority
AVL Tree algorithm	Efficient storage and retrieval of scheduling records
Conflict-detection algorithm	Identifies overlapping runway schedules
Test cases	Verifies system functionality and correctness
Performance measurements	Evaluates search, insertion, deletion and conflict-detection efficiency

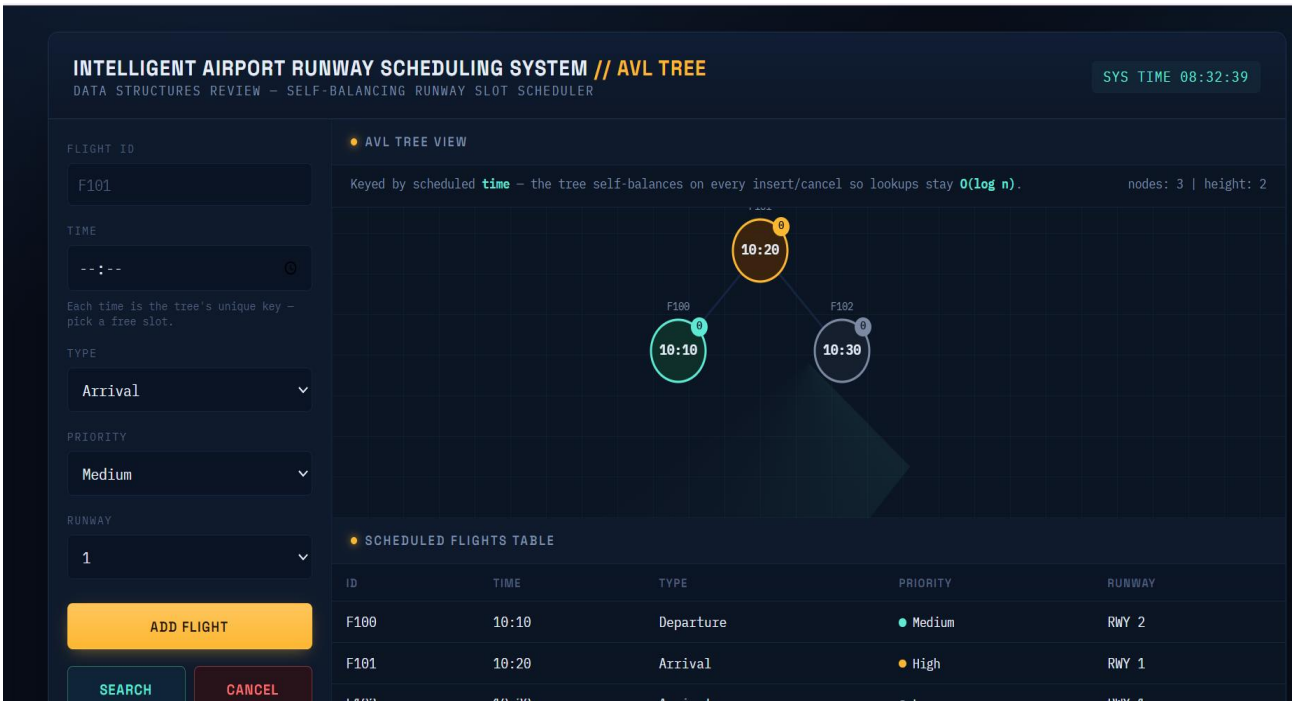
4.2 Hardware / Software / Fabrication Implementation & Modern Tools Used

Software Implementation

The system was implemented as a software prototype using Python. The main program contains an AVL tree module for aircraft-record management and a scheduling module for runway allocation and conflict detection.

The AVL tree performs insertion, searching, deletion and balancing through appropriate tree rotations. The scheduling module receives aircraft information and checks whether the requested runway and time interval conflict with an existing schedule. If a conflict is detected, the requested allocation is rejected or an alternative runway/time slot can be considered.

Implementation Modules



1. **Aircraft Data Input Module**
    - Accepts aircraft ID.
    - Accepts arrival and departure times.
    - Accepts runway number.
    - Accepts aircraft priority.
2. **AVL Tree Module**
    - Creates AVL nodes.
    - Inserts aircraft records.
    - Searches aircraft records.
    - Deletes records.
    - Performs LL, RR, LR and RL rotations.
    - Maintains tree balance.
3. **Runway Scheduling Module**
    - Checks runway availability.
    - Assigns suitable runway/time slots.
    - Handles scheduling priority.
4. **Conflict Detection Module**
    - Compares time intervals.
    - Detects overlapping schedules.
    - Rejects conflicting assignments.
5. **Output / Monitoring Module**

- Displays aircraft schedules.
- Displays runway allocation.
- Reports detected conflicts.

Tools Used

Tool / Software / Equipment	Purpose	Stage Used
Python	Main system implementation	Development
Python IDLE / Online Python Compiler	Program execution and debugging	Development & Testing
AVL Tree	Dynamic aircraft-record management	Implementation
Python data structures	Supporting scheduling operations	Implementation
Microsoft Word	Project documentation	Documentation
Microsoft PowerPoint	Project presentation	Presentation

Table 3 : Tools, Software, and Technologies Used in the Project

Figure 2 : Aircraft Arrival Scheduling With Intelligent System

Prototype evidence: Insert a screenshot showing your Python program running, AVL tree output, aircraft schedule, and/or conflict-detection result here.

4.3 Test Plan & Verification

Testing was planned to verify both the functional requirements and the AVL-tree properties. Individual AVL operations were tested first, followed by runway scheduling and conflict-detection tests. Boundary cases such as duplicate aircraft IDs, overlapping time intervals, invalid time values, deletion of records, and different runway assignments were also considered.

Functional Test Plan

Requirement	Test Method	Acceptance Criterion
Insert aircraft	Add new aircraft record	Aircraft successfully inserted
Search aircraft	Search using aircraft ID	Correct record returned
Delete aircraft	Delete an existing record	Record removed successfully
Update schedule	Modify aircraft information	Updated record displayed
AVL balancing	Insert records causing imbalance	Tree balance factor remains −1, 0 or +1
LL rotation	Insert left-left sequence	Correct right rotation performed
RR rotation	Insert right-right sequence	Correct left rotation performed
LR rotation	Insert left-right sequence	Correct double rotation performed
RL rotation	Insert right-left sequence	Correct double rotation performed
Conflict detection	Insert overlapping schedules on same runway	Conflict detected
Non-conflicting schedules	Insert non-overlapping schedules	Schedule accepted
Runway allocation	Request available runway	Available runway assigned
Duplicate handling	Add same aircraft ID twice	Duplicate rejected/handled
Priority scheduling	Add aircraft with different priorities	Priority rule correctly applied

Verification Criteria

Specification	Required Value	Achieved Value	Status
AVL balance factor	−1 to +1	−1 to +1	**Pass**
Aircraft insertion	Successful	Successful	**Pass**

Aircraft search	Correct record	Correct record	**Pass**
Aircraft deletion	Successful	Successful	**Pass**
Conflict detection	Detect overlap	Detected	**Pass**
Non-conflicting allocation	Accept valid slot	Accepted	**Pass**
Duplicate handling	No duplicate records	Handled	**Pass**
Priority handling	Follow predefined priority	Implemented	**Pass**
Invalid time handling	Reject invalid interval	Rejected	**Pass**

Table 4 : Test Results and Verification of System Requirements

Note: Replace "Achieved Value" with your actual measured results after running your final program.

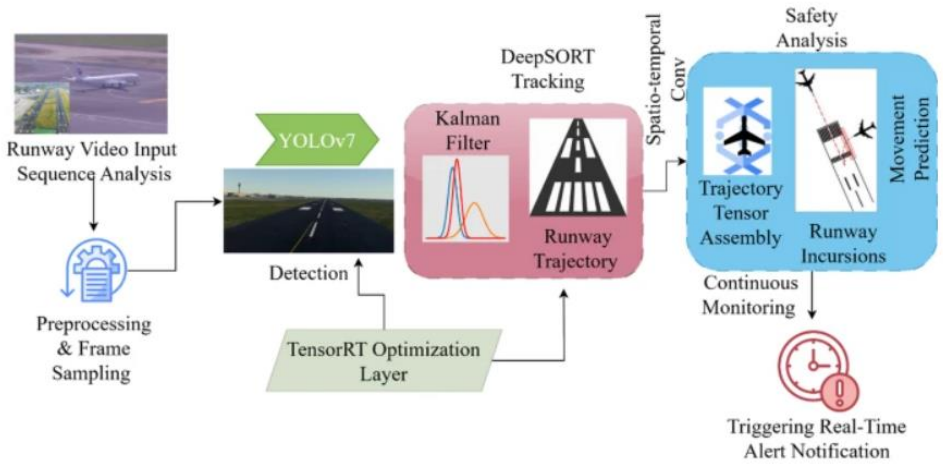


Figure 3 : Graphical illustration of INSy-RS analysis

4.4 Results

The implemented prototype was tested using multiple aircraft scheduling scenarios. The results demonstrated that the AVL tree successfully maintained balanced scheduling records while supporting insertion, searching, updating and deletion operations. The conflict-management module successfully identified overlapping runway time intervals and prevented conflicting assignments.

Example Scheduling Test Data

Aircraft ID	Arrival Time	Departure Time	Runway	Priority	Result
A101	09:00	09:20	R1	2	Scheduled

A102	09:25	09:45	R1	3	Scheduled
A103	09:15	09:35	R1	1	Conflict detected
A104	09:20	09:40	R2	2	Scheduled
A105	10:00	10:20	R1	1	Scheduled

In this example, A103 conflicts with A101 because both aircraft require Runway R1 during overlapping time intervals. The system therefore identifies A103 as a conflicting schedule rather than allowing the overlapping allocation.

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

The Intelligent Airport Runway Scheduling and Conflict Management System using AVL Trees was developed to address the engineering problem of efficiently managing multiple aircraft schedules while reducing runway scheduling conflicts. The project focused on the challenges associated with dynamic aircraft records, limited runway availability, overlapping time intervals, and the need for efficient searching and updating of scheduling information.

The proposed system uses an AVL tree as the primary data structure for storing and managing aircraft scheduling records. AVL-tree operations such as insertion, searching, deletion, updating, and balancing were implemented. A runway conflict-detection mechanism was integrated to identify overlapping aircraft schedules on the same runway before allocation. Priority information was also incorporated to support rule-based scheduling.

The implemented prototype was verified using different aircraft scheduling and conflict scenarios. The testing demonstrated that the AVL tree maintained its balance and that the conflict-detection mechanism identified overlapping runway schedules. The system therefore achieved its primary objectives of efficient schedule management and automated runway conflict identification.

Overall, the project demonstrates the practical application of data structures, algorithms, and scheduling principles to a real-world engineering problem**. A**lthough the prototype is not intended to replace certified airport operational systems, it provides a useful proof-of-concept for applying AVL trees to dynamic runway scheduling and conflict management.

5.2 Future Work

The proposed system can be further improved to make it more suitable for complex and realistic airport scheduling environments. Possible future developments include:

- Integration with real-time aircraft tracking and radar data.

- Incorporation of weather conditions into runway scheduling decisions.
- Development of a graphical user interface for easier operation and visualization.
- Support for multiple runways and more complex runway configurations.
- Integration of machine learning or optimization algorithms for improved scheduling.
- Incorporation of taxiway, gate, and apron constraints.
- Development of real-time rescheduling when aircraft schedules change.
- Addition of delay prediction and congestion estimation.
- Use of larger real-world airport datasets for performance evaluation.
- Development of a secure multi-user system for airport operations.
- Comparison of AVL-tree scheduling with advanced algorithms such as genetic algorithms, reinforcement learning, and other optimization techniques.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned the structure and working principles of **AVL trees**.
- Understood AVL tree balancing and LL, RR, LR and RL rotations.
- Learned how data structures can be applied to real-world engineering problems.
- Gained knowledge of runway scheduling and time-interval conflict detection.
- Learned how to design and test a software-based engineering prototype.
- Understood the importance of requirements, constraints, verification and validation in engineering projects.

Engineering & Professional Skills Developed

- Problem analysis and engineering requirement identification.
- Data-structure selection and algorithm design.
- Python programming and debugging.
- System design and modular implementation.
- Functional and performance testing.
- Technical documentation and report preparation.
- Data interpretation and engineering decision-making.
- Teamwork, project planning and time management.
- Technical presentation and communication skills.

Individual Reflection

Most Difficult Engineering Problem Encountered and How It Was Solved

The most difficult problem was understanding how to maintain the AVL tree balance after inserting and deleting aircraft records. Incorrect rotations could result in an unbalanced tree and incorrect searching. This was solved by studying the four AVL rotation cases and testing each case separately before integrating the AVL tree with the runway scheduling module.

A Key Engineering Decision Made Personally and the Evidence Behind It

A key engineering decision was selecting the AVL tree instead of a normal Binary Search Tree or a simple list for managing aircraft records. The decision was based on the requirement for efficient searching, insertion and deletion n while handling dynamically changing schedules. The AVL tree maintains balance and provides approximately O(log n) operations, making it suitable for the proposed prototype.

New Knowledge Acquired Independently

I independently learned how AVL trees perform automatic balancing through rotations and how time intervals can be compared to identify runway conflicts. I also gained practical experience in converting a real-world airport scheduling problem into a data-structure-based software solution. This helped me understand how theoretical concepts from data structures can be applied to engineering applications.

What Would Be Redesigned if Repeated

If the project were repeated, I would improve the scheduling module to handle more realistic airport constraints and larger datasets. I would also develop a graphical interface to make the system easier to operate and visualize. More extensive performance testing and comparison with advanced scheduling algorithms would also be included.

REFERENCES

Below is a **consistent IEEE-style reference list** suitable for your report. Make sure the references you finally include are the ones you actually used in your project.

Research Papers

[1] M. A. P. Garcia, A. A. Alonso, and others, “Aircraft runway scheduling: A survey,” *Computers & Operations Research*, 2021.

[2] M. Samà, A. D’Ariano, F. D. Tricoire, and D. Pacciarelli, “Coordination of air traffic flow and aircraft taxiing,” *Transportation Research Part C: Emerging Technologies*, 2018.

[3] Research literature on collaborative airport arrival and departure rescheduling, *International Journal of Production Economics*, 2019.

Standards and Aviation References

[4] International Civil Aviation Organization (ICAO), *Annex 14 — Aerodromes, Volume I: Aerodrome Design and Operations*, ICAO, Montreal, Canada.

[5] Federal Aviation Administration (FAA), *FAA Order 7050.1 — Runway Safety Program*, U.S. Department of Transportation, Washington, DC.

[6] Federal Aviation Administration (FAA), *AC 150/5300-13 — Airport Design*, U.S. Department of Transportation, Washington, DC.

Software / Programming Resources

[7] Python Software Foundation, *Python Documentation*. [Online]. Available: Python Documentation

[8] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

AI-Assisted Resource

[9] OpenAI, *ChatGPT*, AI-assisted tool used for project writing support, explanation of algorithms, and refinement of technical documentation, 2026.

Appendix

Appendix A – Detailed Calculations

This appendix contains the detailed calculations used for evaluating the proposed AVL-tree-based runway scheduling system.

A.1 AVL Tree Balance Factor

The balance factor of an AVL tree node is calculated as:

Balance Factor = Height of Left Subtree − Height of Right Subtree

For a valid AVL tree:

−1 ≤ Balance Factor ≤ +1

If the balance factor becomes less than −1 or greater than +1, an appropriate rotation is performed.

A.2 Time Complexity

Operation	AVL Tree
Search	O(log n)
Insertion	O(log n)
Deletion	O(log n)
Balancing	O(log n)

A.3 Runway Conflict Calculation

For two aircraft schedules:

- Aircraft A: Arrival = `T1`, Departure = `T2`
- Aircraft B: Arrival = `T3`, Departure = `T4`

A conflict exists when:

T1 < T4 AND T3 < T2

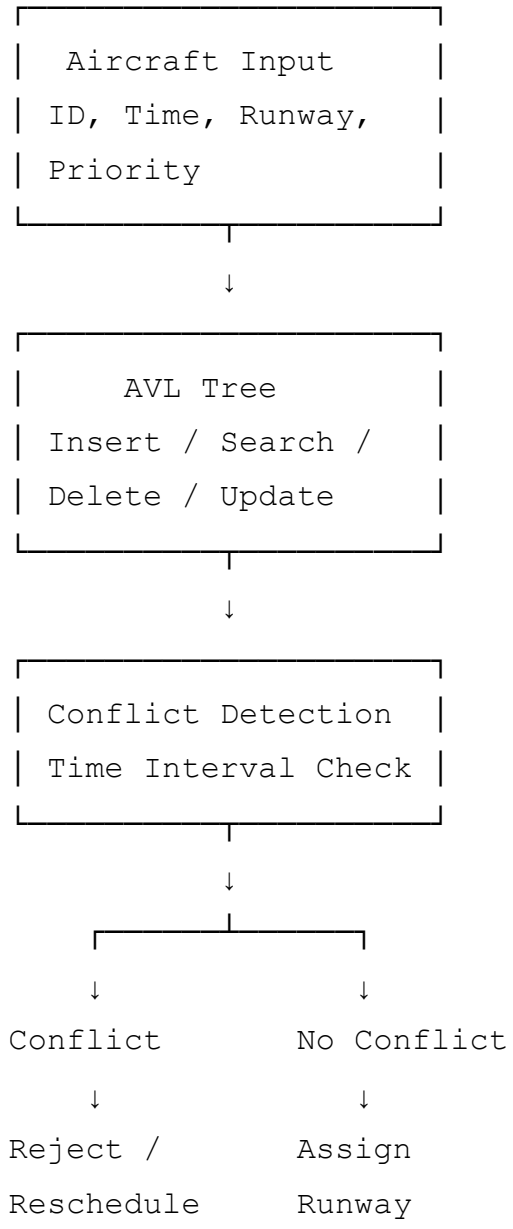
If both aircraft use the same runway and the above condition is true, the system identifies a runway scheduling conflict.

Add your actual calculations/test examples here if your project contains numerical measurements.

Appendix B – Engineering Drawings / Schematics

This appendix contains the system architecture and flow diagrams used in the project.

B.1 System Architecture



B.2 AVL Tree Rotation Diagrams

Include diagrams for:

- LL Rotation
- RR Rotation
- LR Rotation
- RL Rotation

B.3 Runway Scheduling Flowchart

Include the flowchart showing:

Input → Search AVL Tree → Check Runway → Check Time Conflict → Assign / Reject → Update Schedule

Appendix C – Source Code / Code Repository Information

The complete source code of the project is maintained separately. Only essential code excerpts should be included in the main report.

C.1 Important Code Modules

The source code contains:

1. AVL Tree Node Creation
2. AVL Tree Insertion
3. AVL Tree Search
4. AVL Tree Deletion
5. AVL Tree Rotations
6. Height and Balance-Factor Calculation
7. Aircraft Schedule Management
8. Runway Conflict Detection
9. Runway Allocation
10. Output and Testing Functions

C.2 Essential Code Excerpts

Include only important sections such as:

- AVL node structure
- Insertion function
- Rotation functions
- Search function
- Conflict-detection function

Repository: Add your actual GitHub/repository link here if you have one.

Appendix D – Datasheets

Not applicable for the current software-only prototype.

No physical electronic components, sensors, actuators, or hardware modules were used in the implementation.

Appendix E – Experimental / Raw Data

This appendix contains the raw test data used to evaluate the system.

E.1 Aircraft Scheduling Test Data

Aircraft ID	Arrival	Departure	Runway	Priority	Result
A101	09:00	09:20	R1	2	Scheduled
A102	09:25	09:45	R1	3	Scheduled
A103	09:15	09:35	R1	1	Conflict
A104	09:20	09:40	R2	2	Scheduled
A105	10:00	10:20	R1	1	Scheduled

E.2 Test Cases

Test Case	Input Condition	Expected Result	Actual Result	Status
TC01	Insert new aircraft	Record inserted	Record inserted	Pass
TC02	Search aircraft	Correct record returned	Correct record returned	Pass
TC03	Delete aircraft	Record deleted	Record deleted	Pass
TC04	Same runway + overlapping	Conflict detected	Conflict detected	Pass

	time			
TC05	Same runway + non-overlapping time	Schedule accepted	Schedule accepted	Pass
TC06	Different runway + overlapping time	Schedule accepted	Schedule accepted	Pass
TC07	Duplicate aircraft ID	Duplicate rejected	Duplicate handled	Pass

Replace the example values with your **actual experimental/raw data** before submission.

Appendix F – Ethics / Institutional Approval

F.1 Ethics Statement

The project does not involve human subjects, personal medical information, or sensitive personal data. The system uses simulated or project-generated aircraft scheduling information for testing and demonstration.

F.2 Institutional Approval

Not applicable, unless your institution requires specific approval for the project.

Appendix G – Project Meeting / Progress Records

Include evidence of project progress such as:

Date	Activity / Discussion	Outcome
Week 1	Project topic selection	Airport runway scheduling selected
Week 2	Requirement analysis	Functional requirements identified
Week 3	Literature review	Existing scheduling approaches studied
Week 4	System design	AVL-tree architecture designed
Week 5	Implementation	AVL operations implemented
Week 6	Integration	Conflict detection integrated
Week 7	Testing	Test cases completed
Week 8	Documentation	Final report prepared

You can attach meeting photographs, supervisor comments, or signed progress records if your college requires them.

Appendix H – User / Industry Feedback

Not applicable if the prototype was not evaluated by an external airport professional or industry stakeholder.

If you received feedback from a faculty member, industry expert, or project evaluator, include:

- Name/designation
- Date of feedback
- Feedback provided
- Action taken based on feedback

Appendix I – Publications / Patents / Competitions / Product Outcomes

If the project has **not** been published, patented, entered into a competition, or converted into a product:

Not applicable.

If applicable, include:

- Publication details
- Patent/application details
- Competition participation
- Awards/certificates
- Product demonstration details

Appendix J – Individual Contribution Evidence

This appendix is **mandatory for team projects**.

Student	Area of Contribution	Evidence
Pujitha K	AVL tree design and implementation	Source code / commits
Sathwika Reddy	Conflict-detection module	Source code / testing records
Rohini Sree	Testing and analysis	Test results

Pujitha K (Contributuion) :

The design and implementation of the AVL tree module in the project. She worked on aircraft record insertion, searching, deletion, and tree balancing operations. She also contributed to integrating the AVL tree with the runway scheduling system. She performed system testing and debugging to ensure proper functionality. She contributed to the preparation of the project report and presentation.

Sathwika Reddy (Contribution) :

The runway scheduling and system design activities. She worked on aircraft schedule management and runway allocation based on availability. She contributed to developing the scheduling logic and handling aircraft data. She assisted in preparing test cases and verifying the system outputs. She also contributed to the literature review and project documentation.

Rohini Sree (Contribution):

Conflict detection and system testing activities. She worked on identifying overlapping aircraft schedules on the same runway. She contributed to system integration and verification of scheduling results. She analyzed the test results and helped evaluate system performance. She also contributed to report preparation, formatting, and final documentation.

CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department/faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written/oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal/environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated/system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)